

Web Technologies

Detailed Semester Exam Notes | HTML, CSS, JavaScript, Node.js, MongoDB and Full Stack

Prepared for Rahul Yadav. These notes cover WWW, HTTP, HTML5, CSS, JavaScript, jQuery, Node.js, Express, MongoDB, AngularJS, React, PHP, hosting, CMS and full-stack case study according to the provided syllabus.

Unit	Main Area	Exam Focus
I	Web 3.0 and HTML	WWW, HTTP, HTML tags, forms, HTML5, validators
II	CSS	Types, selectors, box model, positioning, menu design
III	JavaScript and jQuery	Variables, operators, loops, events, arrays, objects, functions, UI
IV	Node.js, AngularJS, React, PHP	Server-side JS, Express, MongoDB, frameworks, PHP forms
V	More web technologies	Website working, client-server scripting, FTP, hosting, CMS, full stack

Exam tip: in web technology answers, include small code snippets and diagrams. For HTML/CSS/JS questions, one correct example often makes the answer stronger than plain theory.

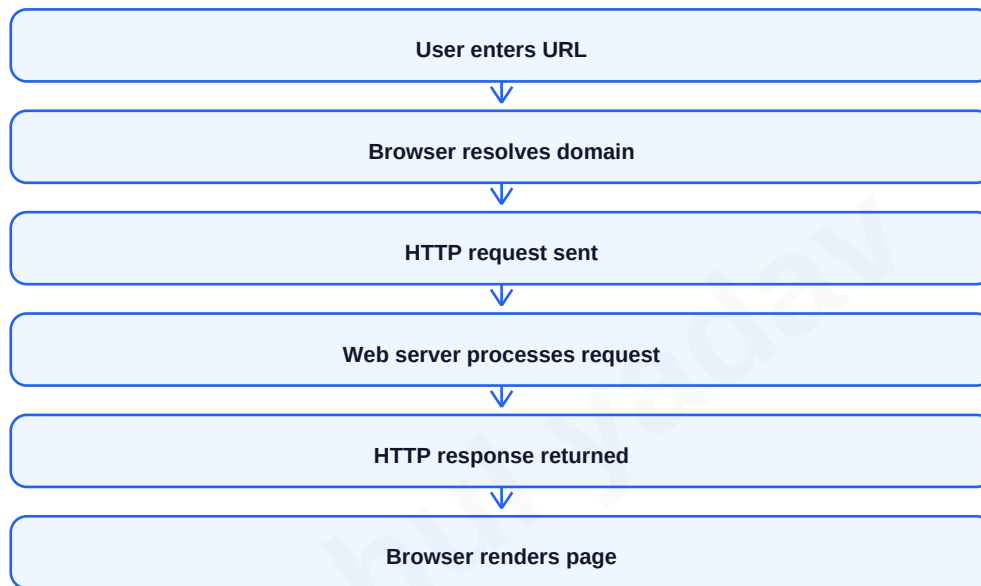
UNIT I: WWW, Internet and Web 3.0

The Internet is a global network of interconnected computer networks that communicate using protocols such as TCP/IP. The World Wide Web is a service on the Internet that provides linked documents, media and applications through web browsers.

WWW works through client-server architecture. The client is usually a web browser and the server stores web pages or web applications. The browser sends an HTTP request and the server returns an HTTP response.

Web 2.0 focuses on user-generated content, social interaction and dynamic web applications. Web 3.0 focuses on intelligent, semantic, decentralized and data-driven web experiences.

- Internet is infrastructure; WWW is a service running on it.
- Web browser requests and displays web resources.
- Web server stores and serves websites/web apps.
- Web 2.0 includes blogs, social media and AJAX apps.
- Web 3.0 includes semantic web, AI-driven services and decentralization.



Term	Meaning
Internet	Global network of networks
WWW	Collection of linked web resources
URL	Address of web resource
Browser	Client software for accessing web
Web Server	Software/machine serving web resources

HTTP Protocol: Request and Response

HTTP stands for HyperText Transfer Protocol. It is an application-layer protocol used for communication between web browser and web server. HTTP is stateless, meaning every request is independent.

An HTTP request contains method, URL/path, headers and optional body. An HTTP response contains status code, headers and optional body. Common methods are GET, POST, PUT, PATCH and DELETE.

HTTPS is HTTP over TLS/SSL and provides encryption, authentication and integrity. Modern websites should use HTTPS for security.

- GET requests data from server.
- POST submits data to server.
- Status code 200 means OK.
- Status code 404 means resource not found.
- Status code 500 means server error.

Method/Status	Meaning	Example
GET	Read resource	Open page/search
POST	Submit data	Login/register form
PUT	Replace/update resource	Update profile
200	Successful response	Page loaded
404	Not found	Wrong URL
500	Server error	Backend exception

```
GET /index.html HTTP/1.1
Host: example.com
```

```
HTTP/1.1 200 OK
Content-Type: text/html
```

Markup Language and Basic HTML Structure

A markup language uses tags to describe the structure and presentation of text. HTML stands for HyperText Markup Language and is used to create web pages. HTML elements usually have opening tag, content and closing tag.

A basic HTML document contains doctype declaration, html element, head section and body section. The head section contains metadata, title, links to CSS and scripts. The body section contains visible content.

HTML is not a programming language because it does not contain logic like loops and conditions. It structures content for browsers.

- tells browser to use HTML5 standard mode.
- is root element.
- contains metadata and linked resources.
- contains visible content.
- HTML tags should be nested properly.

Part	Purpose
DOCTYPE	Defines HTML version
head	Metadata, title, links, scripts
body	Visible page content
title	Browser tab title
meta	Character set, viewport and SEO data

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <meta charset="UTF-8">
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Head Section, Meta Tags, CSS and Script Tags

The head section stores information about the document that is not directly displayed as page content. It contains title, meta tags, external stylesheets, internal CSS and script links.

Meta tags provide metadata such as character encoding, viewport settings, description and keywords. The viewport meta tag is important for responsive mobile design.

CSS can be linked using link tag, and JavaScript can be added using script tag. For better loading, JavaScript files are often placed before closing body tag or loaded with defer attribute.

- meta charset defines character encoding.
- meta viewport helps page fit mobile screens.
- link rel='stylesheet' attaches external CSS.
- style tag writes internal CSS.
- script tag loads or writes JavaScript.

Tag	Purpose	Example
title	Page title	<title>Home</title>
meta	Metadata	charset, viewport
link	External CSS/file relation	style.css
style	Internal CSS	CSS rules
script	JavaScript	app.js

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link rel="stylesheet" href="style.css">
<script src="app.js" defer></script>
```

Common HTML Tags: Text, Links, Images and Div

HTML provides many tags to structure web content. Heading tags h1 to h6 define headings. Paragraph tag p defines text paragraphs. Anchor tag a creates hyperlinks. Image tag img displays images.

The div tag is a block-level container used to group elements for layout and styling. The span tag is an inline container used for styling part of text.

Semantic tags such as header, nav, main, section, article and footer describe meaning of content and improve readability, accessibility and SEO.

- Anchor tag uses href attribute.
- Image tag uses src and alt attributes.
- div is block-level; span is inline.
- alt text improves accessibility.
- Semantic tags are preferred over excessive div usage.

Tag	Use
h1-h6	Headings
p	Paragraph
a	Hyperlink
img	Image
div	Block container
span	Inline container
ul/ol/li	Lists
br/hr	Line break / horizontal rule

```
<a href="https://example.com">Visit Site</a>

<div class="card">
  <h2>Profile</h2>
  <p>Welcome Rahul</p>
</div>
```

Tables, Object, Iframe and Multimedia Tags

HTML tables display data in rows and columns. Main table tags are table, tr, th, td, caption, thead, tbody and tfoot. Tables should be used for tabular data, not page layout.

The object tag embeds external resources such as PDF or multimedia. The iframe tag embeds another web page inside current page. Multimedia tags audio and video are used in HTML5.

Earlier websites used tables for layout, but modern websites use CSS layout techniques such as flexbox and grid.

- table creates table structure.
- tr defines row, th defines header cell and td defines data cell.
- iframe embeds external page/map/video.
- object embeds external content.
- Use CSS for layout instead of tables.

Tag	Purpose
table	Create table
tr	Table row
th	Header cell
td	Data cell
caption	Table title
iframe	Embed another page
audio/video	Embed media

```
<table>
  <tr><th>Name</th><th>Marks</th></tr>
  <tr><td>Rahul</td><td>85</td></tr>
</table>
```

HTML Forms and Form Attributes

HTML forms collect user input and send it to a server for processing. The form tag contains input controls such as text boxes, textarea, radio buttons, checkboxes, dropdowns, file upload and submit button.

Important form attributes are action, method, enctype, target and autocomplete. action defines server URL. method defines GET or POST. enctype is important for file uploads.

Forms are central to login pages, registration pages, search forms, feedback forms and data entry applications.

- GET sends data in URL query string and is suitable for search.
- POST sends data in request body and is suitable for sensitive or large data.
- name attribute is required to submit input value.
- required attribute performs basic validation.
- multipart/form-data is used for file upload.

Control	HTML Element	Use
Text Input	input type='text'	Single line text
Textarea	textarea	Multiline text
Radio	input type='radio'	Choose one option
Checkbox	input type='checkbox'	Choose multiple options
Dropdown	select/option	Choose from list
File	input type='file'	Upload file
Hidden	input type='hidden'	Send invisible value

```
<form action="/submit" method="post">
  <input type="text" name="username" required>
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
```

HTML Validators and HTML5 Features

HTML validation checks whether HTML code follows standards. Validators detect missing tags, wrong nesting, invalid attributes and accessibility issues. W3C Markup Validator is a common tool.

HTML5 introduced semantic elements, audio/video support, canvas, SVG support, local storage, session storage, geolocation, form validation, new input types and simplified doctype.

HTML5 doctype is simply . It tells browser to render page in standards mode.

- Validators improve browser compatibility.
- Semantic tags make page structure meaningful.
- Canvas is used for drawing graphics using JavaScript.
- Local storage stores key-value data in browser.
- New input types include email, date, number, range and color.

HTML5 Feature	Purpose
Semantic tags	header, nav, main, section, article, footer
Audio/Video	Native media support
Canvas	Dynamic drawing
Local Storage	Browser-side persistent storage
Form validation	required, pattern, email, number
Geolocation	Access user location with permission

UNIT II: CSS Introduction and Types of CSS

CSS stands for Cascading Style Sheets. It is used to control presentation of HTML pages including color, font, spacing, layout, borders, background and responsive behavior.

There are three types of CSS: inline CSS, internal CSS and external CSS. Inline CSS is written in style attribute. Internal CSS is written inside style tag. External CSS is written in separate .css file and linked to HTML.

External CSS is preferred for real projects because it separates content from presentation and allows reuse across multiple pages.

- CSS improves visual design and consistency.
- Cascading means multiple rules can apply and priority decides final style.
- External stylesheet improves maintainability.
- Inline CSS has high priority but poor maintainability.
- CSS can make web pages responsive.

Type	Location	Use
Inline CSS	style attribute	Quick one-element styling
Internal CSS	style tag in head	Single page styling
External CSS	.css file	Reusable site-wide styling

```
/* External CSS */
h1 {
  color: navy;
  font-size: 28px;
}
```

CSS Selectors

CSS selectors select HTML elements for styling. Common selectors are universal selector, type selector, id selector, class selector, descendant selector, child combinator and pseudo-class selector.

The id selector selects one unique element using #id. The class selector selects one or more elements using .class. Type selector selects all elements of a tag name.

Specificity decides which CSS rule applies when multiple rules target the same element. Inline style has highest specificity, then id, class and element selectors.

- Universal selector * selects all elements.
- Type selector targets tag names like p or h1.
- Class selector is reusable.
- ID selector should be unique.
- Child combinator > selects direct children only.

Selector	Example	Meaning
Universal	*	All elements
Type	p	All paragraphs
Class	.menu	Elements with class menu
ID	#header	Element with id header
Descendant	div p	p inside div
Child	ul > li	direct li children
Pseudo-class	a:hover	hover state

CSS Text, Font, Background and List Properties

CSS text and font properties control how text appears. Important properties are color, font-family, font-size, font-weight, text-align, text-decoration, line-height and letter-spacing.

Background properties control element background. These include background-color, background-image, background-repeat, background-position, background-size and background-attachment.

List properties control ordered and unordered lists. list-style-type changes marker style and list-style-position controls marker placement.

- Use relative units like rem/em for scalable typography.
- Use line-height for readability.
- background-size: cover scales image to cover area.
- text-decoration: none is commonly used for nav links.
- list-style-type can be disc, circle, square, decimal or none.

Property Group	Examples
Text	color, text-align, text-transform, text-decoration
Font	font-family, font-size, font-weight, line-height
Background	background-color, image, repeat, size
List	list-style-type, position, image
Display	block, inline, inline-block, none, flex, grid

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f4f6f8;  
}  
ul { list-style-type: square; }
```

CSS Box Model, Borders and Block Properties

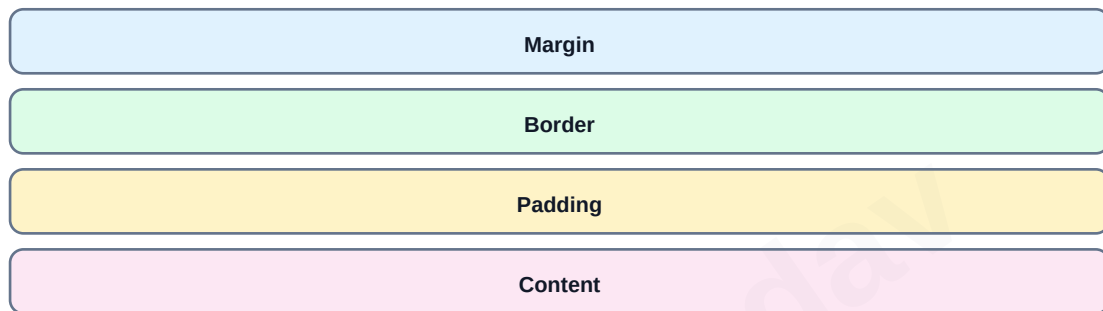
The CSS box model describes how every HTML element is treated as a rectangular box. It consists of content, padding, border and margin. Width and height normally apply to content area unless box-sizing is changed.

Padding is space between content and border. Border surrounds padding and content. Margin is space outside border. box-sizing: border-box makes width include content, padding and border.

Block elements start on a new line and take full available width by default. Inline elements take only required width and do not start on new line.

- content is actual text/image area.
- padding creates inner spacing.
- border draws boundary around element.
- margin creates outer spacing.
- box-sizing: border-box is useful for predictable layouts.

CSS Box Model



Property	Purpose
width/height	Set element size
padding	Inner spacing
border	Element boundary
margin	Outer spacing
display	Block/inline/flex/grid behavior
box-sizing	Controls size calculation

CSS Positioning and Layout

CSS positioning controls where elements appear. The position property can be static, relative, absolute, fixed or sticky. top, right, bottom and left properties work with positioned elements.

Modern layouts are built using flexbox and CSS grid. Flexbox is one-dimensional and useful for rows or columns. Grid is two-dimensional and useful for complex page layouts.

Responsive design uses relative units, media queries and flexible layouts to make pages work on different screen sizes.

- static is default position.
- relative moves element relative to normal position.
- absolute positions element relative to nearest positioned ancestor.
- fixed stays relative to viewport.
- sticky switches between relative and fixed based on scroll.

Position	Meaning	Use
static	Normal document flow	Default
relative	Offset from original position	Small movement
absolute	Positioned ancestor based	Dropdowns/modals
fixed	Viewport based	Fixed header
sticky	Scroll-based sticking	Sticky navbar

```
.container {  
  display: flex;  
  gap: 16px;  
}  
@media (max-width: 600px) {  
  .container { flex-direction: column; }  
}
```

Converting Table Layout to CSS Layout and Menu Design

Old websites used HTML tables to create layouts. This approach mixes content with presentation and is not responsive. Modern websites use CSS layouts such as flexbox, grid and semantic HTML.

CSS menu design uses nav, ul, li and anchor tags styled with CSS. Horizontal menus display list items in a row, while vertical menus display them in a column.

A good menu should be readable, keyboard accessible, responsive and visually clear. Hover and active states improve usability.

- Use semantic nav tag for navigation.
- Use flex-direction: row for horizontal menu.
- Use flex-direction: column for vertical menu.
- Use :hover for hover effect.
- Use media queries to convert desktop menu into mobile layout.

Old Table Layout	Modern CSS Layout
Tables used for page structure	Use header, nav, main, section, footer
Hard to maintain	Reusable CSS classes
Poor responsive behavior	Flexbox/grid responsive design
Mixed content and design	Separation of HTML and CSS

```
nav ul {  
  list-style: none;  
  display: flex;  
  gap: 20px;  
}  
nav a:hover { color: #2563eb; }
```

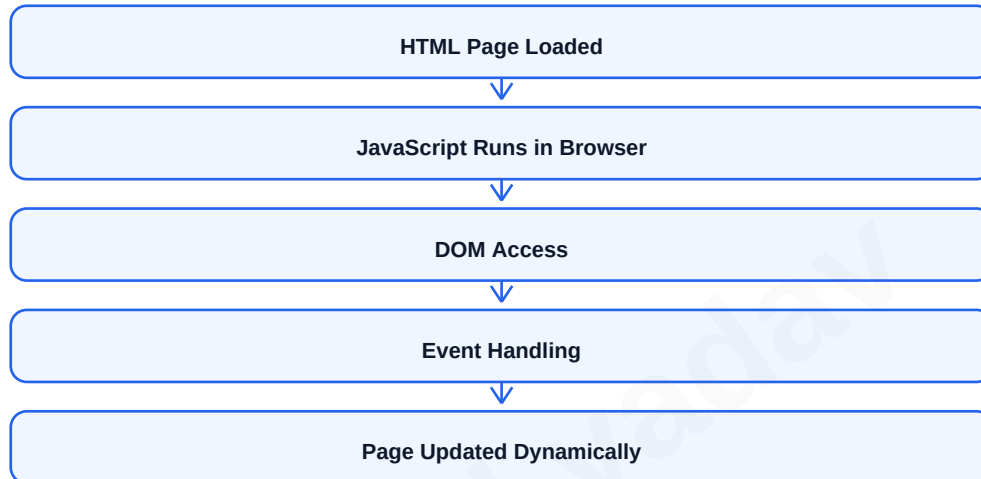
UNIT III: JavaScript and Client-Side Scripting

Client-side scripting means scripts run in the user's browser. JavaScript is the main client-side scripting language used to create dynamic and interactive web pages.

JavaScript can validate forms, handle events, manipulate HTML/CSS, create animations, fetch data from server and update page without reload. It works with the Document Object Model.

JavaScript can be written inline, inside script tag or in an external .js file. External JavaScript is preferred for maintainability.

- JavaScript is interpreted by browser engine.
- It is case-sensitive.
- It supports dynamic typing.
- It can manipulate DOM elements.
- It works asynchronously using callbacks, promises and async/await.



Feature	Meaning
Client-side	Runs in browser
Dynamic typing	Variable type decided at runtime
Event-driven	Responds to clicks, input, load etc.
DOM manipulation	Change page elements
Asynchronous	Can handle delayed operations without blocking

JavaScript Variables, Operators and Conditions

Variables store data values. JavaScript supports var, let and const. let is block-scoped and used for variables that change. const is block-scoped and used for values that should not be reassigned.

Operators perform operations on values. JavaScript has arithmetic, assignment, comparison, logical, string and ternary operators. Strict equality === compares value and type.

Conditional statements execute code based on condition. Common statements are if, if-else, else-if ladder and switch.

- Prefer let and const over var.
- Use === instead of == for safer comparison.
- Logical operators are &&, || and !.
- switch is useful for multiple fixed choices.
- Ternary operator is compact form of if-else.

Operator Type	Examples
Arithmetic	+, -, *, /, %, **
Comparison	>, <, >=, <=, ===, !==
Logical	&&, , !
Assignment	=, +=, -=, *=
Ternary	condition ? a : b

```
const marks = 75;
if (marks >= 60) {
  console.log("First division");
} else {
  console.log("Needs improvement");
}
```


JavaScript Loops, Popup Boxes and Events

Loops execute a block of code repeatedly. JavaScript supports for, while, do-while, for-of and for-in loops. for-of is useful for arrays and iterable objects, while for-in is used for object keys.

Popup boxes include alert, confirm and prompt. alert displays a message, confirm asks OK/Cancel and prompt accepts text input.

Events are actions that happen in browser, such as click, submit, keyup, mouseover and load. Event handlers respond to these actions.

- for loop is useful when number of iterations is known.
- while loop is useful when condition controls loop.
- addEventListener is preferred for attaching events.
- Form submit event is used for validation.
- Prevent default behavior using event.preventDefault().

Concept	Example	Use
for loop	for(let i=0;i<5;i++)	Counted loop
while loop	while(x<10)	Condition loop
alert	alert('Hi')	Message
confirm	confirm('Delete?')	OK/Cancel
event	click, submit	User interaction

```
document.querySelector("#btn").addEventListener("click", function () {  
    alert("Button clicked");  
});
```

Arrays, Objects and Functions in JavaScript

An array stores multiple values in ordered form. JavaScript arrays are dynamic and can store different types of values. Common methods include push, pop, shift, unshift, map, filter, reduce and forEach.

An object stores data as key-value pairs. Objects are used to represent entities such as student, product or user. Properties store data and methods store behavior.

A function is a reusable block of code. JavaScript supports function declarations, function expressions and arrow functions.

- Arrays use zero-based index.
- Objects use property names.
- Functions reduce repetition.
- Arrow functions provide compact syntax.
- map transforms array, filter selects items and reduce combines values.

Topic	Meaning
Array	Ordered collection
Object	Key-value data structure
Function	Reusable code block
Method	Function inside object
Callback	Function passed as argument

```
const student = { name: "Rahul", marks: 85 };
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);

function greet(name) {
  return "Hello " + name;
}
```

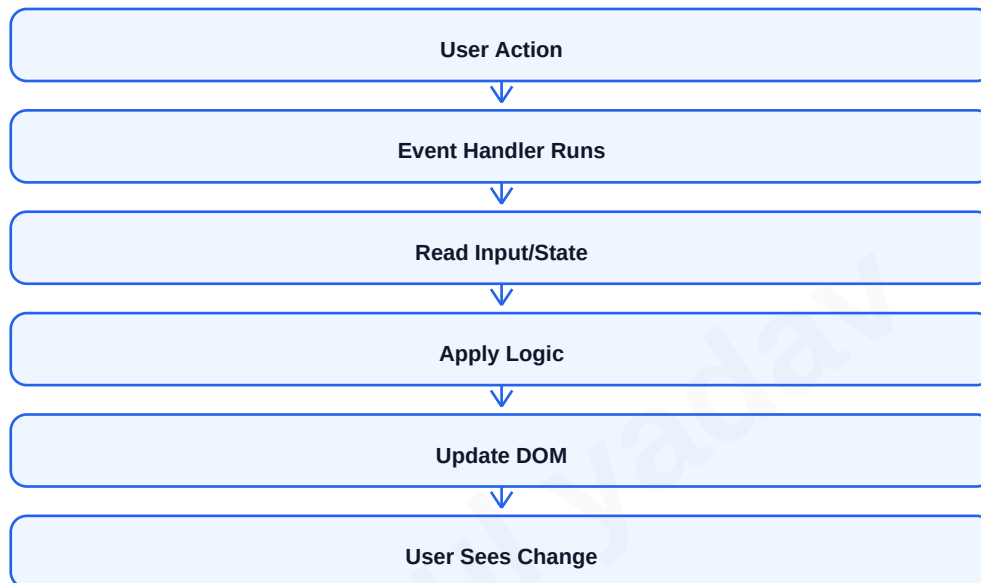
DOM Manipulation and Real-Time JavaScript Examples

DOM stands for Document Object Model. It represents HTML document as a tree of objects. JavaScript can access and modify DOM elements, attributes, styles and content.

Common DOM methods include `getElementById`, `querySelector`, `querySelectorAll`, `createElement`, `appendChild` and `remove`. DOM properties include `innerHTML`, `textContent`, `value`, `classList` and `style`.

Real-time examples include form validation, image slider, dynamic table, calculator, todo list, theme switcher and live character counter.

- `querySelector` selects first matching CSS selector.
- `textContent` changes text safely.
- `innerHTML` can insert HTML but should be used carefully.
- `classList.add/remove/toggle` changes classes.
- `value` reads form input value.



```
const nameBox = document.querySelector("#name");
const output = document.querySelector("#output");
nameBox.addEventListener("input", () => {
  output.textContent = "Hello " + nameBox.value;
});
```

jQuery Introduction, Features and Syntax

jQuery is a JavaScript library designed to simplify DOM manipulation, event handling, animation and AJAX. It became popular because it reduced browser compatibility problems and made JavaScript code shorter.

jQuery uses the \$ symbol as shortcut for jQuery function. Selectors are similar to CSS selectors. A typical jQuery statement selects elements and applies an action.

Although modern JavaScript can do many things directly, jQuery is still important in many legacy and academic web projects.

- jQuery simplifies DOM selection.
- It supports chaining methods.
- It provides event helpers like click and submit.
- It includes animation methods like hide, show and fadeIn.
- It supports AJAX for server communication.

Feature	Meaning
Selectors	Select elements using CSS-like syntax
DOM manipulation	Change content/classes/styles
Events	click, change, submit handlers
Effects	show, hide, fade, slide
AJAX	Load/send data without page reload

```
$(document).ready(function () {  
    $("#btn").click(function () {  
        $("#message").text("Hello jQuery");  
    });  
});
```

jQuery Functions and Form UI Designing

jQuery functions are used for selecting elements, changing CSS, adding/removing classes, handling events, validating forms and creating UI effects. Common functions are html, text, val, css, addClass, removeClass, toggleClass, hide, show and on.

In form UI design, jQuery can show validation messages, highlight invalid fields, enable/disable buttons and dynamically add input fields. It improves user experience when used carefully.

AJAX with jQuery can submit form data without full page reload, which creates smoother web applications.

- val() reads or sets form input value.
- text() sets plain text and html() sets HTML.
- css() changes inline style.
- on() attaches event handler.
- \$.ajax, \$.get and \$.post are used for AJAX requests.

jQuery Function	Purpose
text/html	Read or set content
val	Read or set form value
css	Apply styles
addClass/removeClass	Change classes
hide/show/toggle	Visibility effects
ajax/get/post	Server communication

```
$("#loginForm").on("submit", function (e) {  
    e.preventDefault();  
    const email = $("#email").val();  
    if (email === "") {  
        $("#error").text("Email is required");  
    }  
});
```

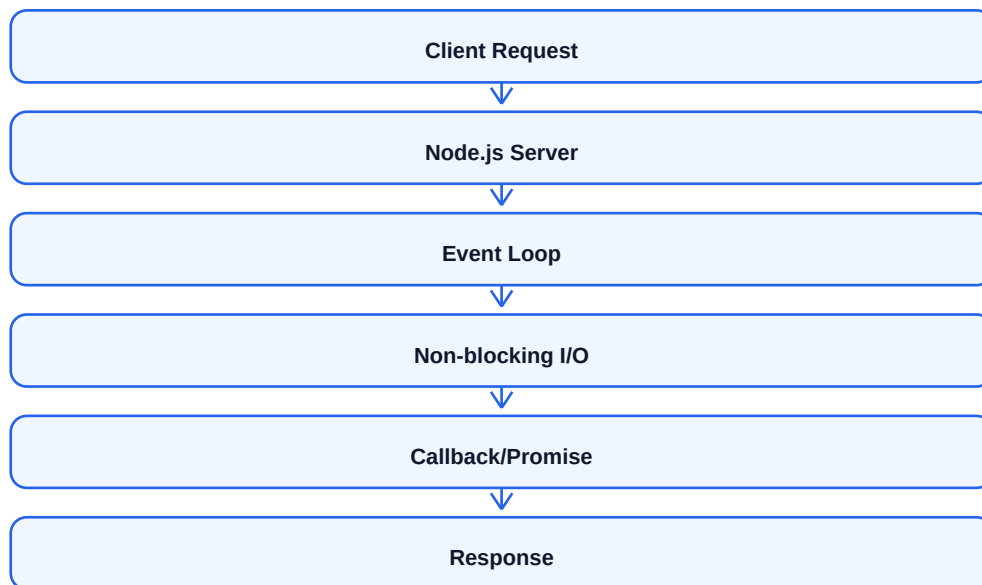
UNIT IV: Node.js and Server-Side JavaScript

Node.js is a runtime environment that allows JavaScript to run outside the browser. It is built on Chrome V8 engine and is widely used for server-side applications, APIs, tools and real-time systems.

Node.js uses non-blocking, event-driven I/O. This makes it efficient for applications with many simultaneous connections, such as chat apps, APIs and streaming services.

npm is Node Package Manager. It is used to install, publish, extend and manage Node.js modules and dependencies.

- Install Node.js from official installer or package manager.
- node command runs JavaScript files.
- npm init creates package.json.
- npm install installs dependencies.
- CommonJS uses require/module.exports; modern Node can use ES modules.



Term	Meaning
Node.js	JavaScript runtime outside browser
npm	Package manager
package.json	Project metadata and dependencies
Module	Reusable code unit
Event Loop	Mechanism handling async operations

Node.js Modules and HTTP Server

A module is a reusable piece of code. Node.js has built-in modules such as http, fs, path and os. Developers can also create custom modules or install third-party modules from npm.

The http module can create a basic web server. It listens on a port and sends response for incoming requests. Express is usually used for real applications because it provides routing and middleware.

Publishing a module means making it available through npm registry. Managing modules includes installing, updating and removing dependencies.

- Core modules come with Node.js.
- Local modules are created by developer.
- Third-party modules are installed using npm.
- http.createServer creates basic server.
- Server listens on a port such as 3000.

Module Type	Example
Core module	http, fs, path
Local module	./utils.js
Third-party module	express, mongoose
Package file	package.json
Dependency install	npm install express

```
const http = require("http");
const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello Node.js");
});
server.listen(3000);
```

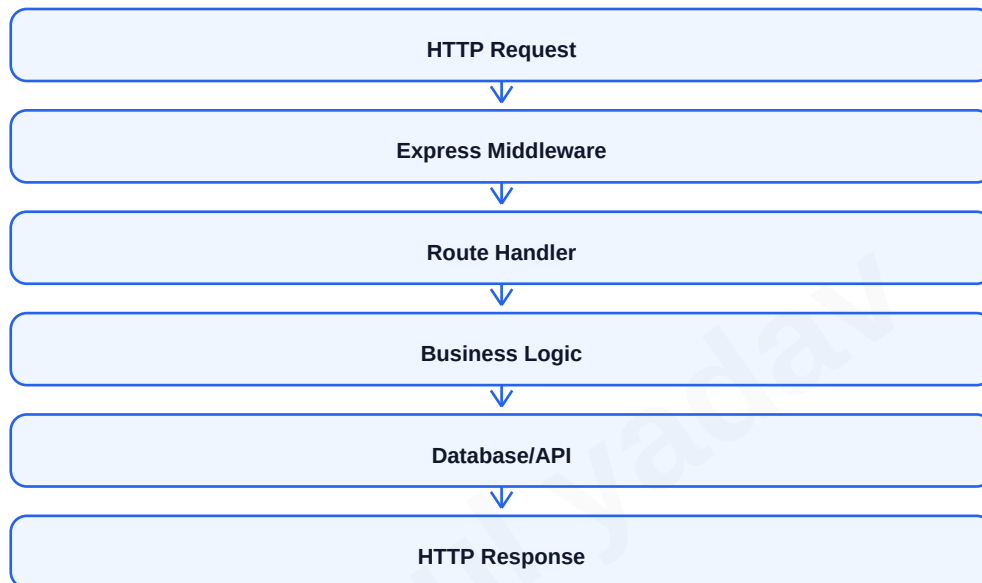
Express Framework

Express is a minimal and flexible Node.js web framework used to build web applications and REST APIs. It provides routing, middleware, request/response handling and template integration.

Routing maps HTTP methods and URL paths to handler functions. Middleware functions run between request and response and can perform logging, authentication, parsing and error handling.

Express greatly reduces boilerplate compared with raw http module.

- app.get handles GET requests.
- app.post handles POST requests.
- express.json parses JSON request body.
- Middleware can modify req/res objects.
- Express is commonly used with MongoDB and Mongoose.



```
const express = require("express");
const app = express();
app.use(express.json());

app.get("/api/users", (req, res) => {
  res.json([{ name: "Rahul" }]);
});
app.listen(3000);
```


MongoDB Overview

MongoDB is a NoSQL document-oriented database. It stores data in flexible JSON-like documents called BSON documents. A collection in MongoDB is similar to a table, and a document is similar to a row.

MongoDB is useful for applications where data structure changes frequently or nested data is common. It works well with JavaScript-based applications because documents resemble JSON objects.

Common operations are insert, find, update and delete. Mongoose is a popular ODM library for using MongoDB with Node.js.

- Database contains collections.
- Collection contains documents.
- Document contains key-value fields.
- MongoDB schema is flexible.
- Mongoose adds schema and model layer to MongoDB.

Relational DB	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	_id
SQL Query	MongoDB query object

```
// MongoDB document example
{
  "_id": 1,
  "name": "Rahul",
  "skills": ["HTML", "CSS", "JS"]
}
```

AngularJS, React and Modern Frontend Frameworks

AngularJS is a JavaScript framework for building dynamic single-page applications. It uses concepts such as modules, controllers, directives, services and two-way data binding.

React is a JavaScript library for building user interfaces using components. It uses virtual DOM and one-way data flow. React applications are built as reusable components.

Modern web applications commonly separate frontend and backend. Frontend frameworks call backend APIs and update UI dynamically.

- AngularJS uses directives like ng-app, ng-model and ng-repeat.
- Two-way binding syncs model and view.
- React component returns UI structure.
- React state controls dynamic UI.
- SPA loads once and updates content without full page reload.

Feature	AngularJS	React
Type	Framework	Library
Data Binding	Two-way	Mostly one-way
Main Unit	Module/controller/directive	Component
DOM	Real DOM based	Virtual DOM
Use	SPA development	Component-based UI

PHP Basics, Syntax, Decisions and Loops

PHP is a server-side scripting language used to create dynamic web pages. PHP code runs on server and generates HTML response for browser. PHP files usually have .php extension.

PHP syntax starts with . Variables begin with \$ sign. PHP supports if-else, switch, for, while, do-while and foreach loops.

PHP can be embedded inside HTML and is widely used with MySQL and CMS platforms such as WordPress.

- PHP runs on server, not browser.
- echo prints output.
- Variables start with \$.
- Form data can be read using \$_GET and \$_POST.
- PHP arrays can be indexed or associative.

Concept	Example
Variable	\$name = 'Rahul';
Output	echo \$name;
Decision	if, else, switch
Loop	for, while, foreach
Array	\$arr = [1,2,3];

```
<?php
$marks = 80;
if ($marks >= 60) {
    echo "First division";
} else {
    echo "Try again";
}
?>
```

PHP with HTML, Arrays, Functions and Form Processing

PHP is often mixed with HTML to generate dynamic content. Server processes PHP code and sends final HTML to browser. This allows pages to display data based on user input or database records.

PHP arrays store multiple values. Indexed arrays use numeric indexes, associative arrays use named keys and multidimensional arrays contain arrays inside arrays.

PHP form processing reads submitted values using \$_GET or \$_POST. Input validation and sanitization are important for security.

- Use method='post' for sensitive form data.
- \$_POST reads POST data.
- \$_GET reads query string data.
- Functions are created using function keyword.
- Always validate and sanitize user input.

```
<form method="post">
  <input name="username">
  <button type="submit">Send</button>
</form>
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    echo "Hello " . htmlspecialchars($_POST["username"]);
}
?>
```

Browser Control, Detection and Strings

Browser control and detection involve identifying browser capabilities and controlling browser behavior through JavaScript or server-side logic. Earlier websites checked browser names, but modern practice prefers feature detection.

Feature detection checks whether a browser supports a particular API before using it. This is better than relying on browser name because browsers change over time.

String handling is important in JavaScript and PHP. Common operations include length, substring, search, replace, split, trim and concatenation.

- Feature detection is safer than browser sniffing.
- navigator.userAgent contains browser information but can be unreliable.
- Strings are immutable in JavaScript.
- PHP uses . operator for string concatenation.
- Input strings should be escaped before displaying in HTML.

String Operation	JavaScript	PHP
Length	str.length	strlen(\$str)
Substring	slice()	substr()
Replace	replace()	str_replace()
Split	split()	explode()
Trim	trim()	trim()
Concatenate	+ or template literal	. operator

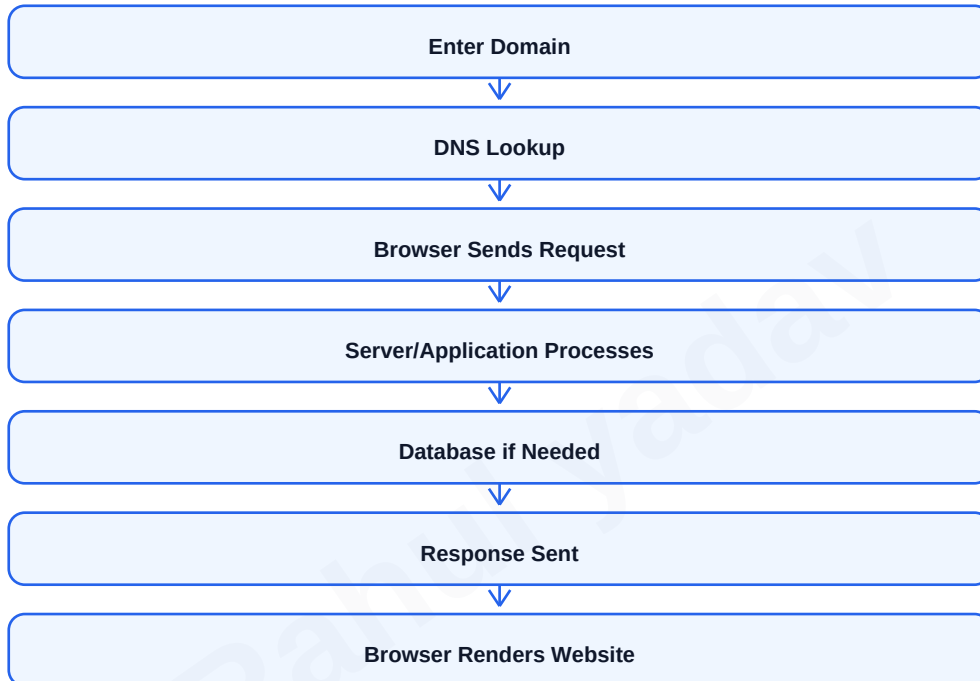
UNIT V: How Website Works

A website works through interaction between client, DNS, server, application code and database. When user enters domain name, DNS converts it into IP address. Browser sends HTTP request to server. Server returns HTML, CSS, JavaScript and other resources.

For dynamic sites, the web server passes request to application server. Application may query database, process business logic and generate response. Browser then renders the response and executes client-side scripts.

Caching, compression and CDN are used to improve performance.

- Domain name maps to IP address using DNS.
- Browser downloads HTML first, then linked CSS/JS/images.
- Static website returns fixed files.
- Dynamic website generates response using server-side code.
- Database stores persistent application data.



Component	Role
Client/Browser	Requests and displays website
DNS	Converts domain to IP
Web Server	Handles HTTP requests
Application Server	Runs backend logic
Database	Stores data
CDN	Delivers static files faster

Client, Server, Uploading, FTP and Hosting

The client is the user's device or browser that requests resources. The server is a computer or software that provides resources or services. Client-server architecture is the base of web applications.

Uploading means transferring website files or data from local machine to server. FTP, SFTP and hosting control panels are common ways to upload files.

Web hosting provides server space and services to publish websites on the Internet. Domain name points users to the hosted website.

- FTP stands for File Transfer Protocol.
- SFTP is secure FTP over SSH.
- Hosting can be shared, VPS, dedicated or cloud.
- Domain connects human-friendly name to server address.
- Static hosting serves HTML/CSS/JS files; dynamic hosting runs backend code.

Hosting Type	Meaning	Use
Shared Hosting	Many sites on one server	Small websites
VPS	Virtual private server	Medium apps
Dedicated	Whole server for one client	High traffic/control
Cloud Hosting	Scalable cloud resources	Modern apps
Static Hosting	Only static files	Portfolio/docs

Client-Side and Server-Side Scripting Languages

Client-side scripting runs in browser and improves interactivity. JavaScript is the main client-side scripting language. It handles DOM manipulation, validation, events and API calls.

Server-side scripting runs on server and handles business logic, database operations, authentication and response generation. Examples include PHP, Node.js, Python, Java and Ruby.

Modern applications often use both: client-side JavaScript for UI and server-side code for data processing and security.

- Client-side code is visible to users, so sensitive logic should not be placed there.
- Server-side code can securely access databases.
- Client-side validation improves UX but server-side validation is mandatory.
- AJAX/fetch connects frontend with backend APIs.
- Full-stack development includes both client and server work.

Basis	Client-Side Scripting	Server-Side Scripting
Runs On	Browser	Server
Main Language	JavaScript	PHP, Node.js, Python, Java
Use	UI interaction and validation	Database and business logic
Security	Visible to user	Hidden from user
Output	DOM updates/API calls	HTML/JSON response

CMS: WordPress, Joomla and Drupal

CMS stands for Content Management System. It allows users to create, edit, organize and publish website content without writing code for every page.

WordPress, Joomla and Drupal are popular CMS platforms. WordPress is easiest and widely used for blogs, business websites and e-commerce. Joomla provides more built-in structure. Drupal is powerful and flexible for complex websites.

CMS platforms use themes for design and plugins/extensions/modules for extra functionality.

- CMS separates content management from coding.
- Themes control appearance.
- Plugins/extensions add features.
- CMS usually uses database to store content.
- Security updates are important for CMS websites.

CMS	Strength	Common Use
WordPress	Easy, many themes/plugins	Blogs, business websites
Joomla	Flexible content management	Portals/community sites
Drupal	Powerful and secure	Complex enterprise/government sites
Custom CMS	Built for specific needs	Special business workflows

Full Stack Web Development Case Study

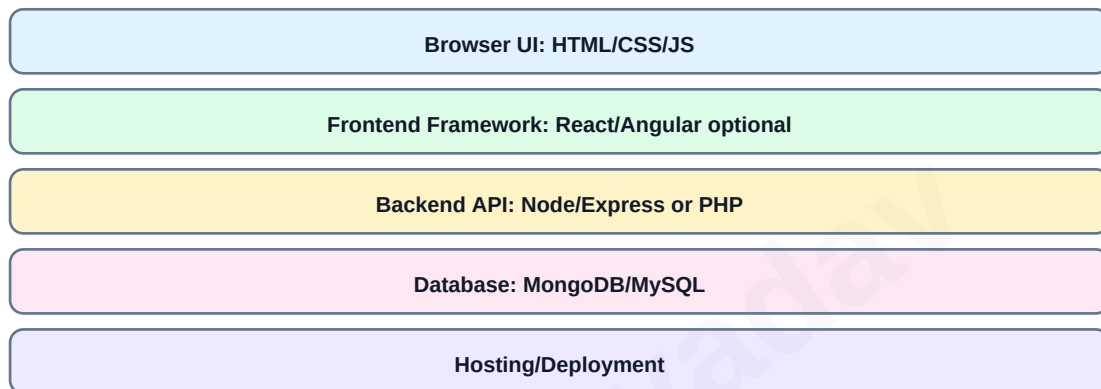
Full stack development means building both frontend and backend parts of a web application. A typical full-stack app has UI, client-side logic, server API, database and deployment setup.

Example case study: Student Management System. Frontend pages allow adding, viewing, editing and deleting students. Backend API handles requests. Database stores student records. Authentication protects admin routes.

The development flow starts with requirements, UI design, database schema, API design, frontend implementation, backend implementation, testing and deployment.

- Frontend: HTML, CSS, JavaScript or React/Angular.
- Backend: Node.js/Express or PHP.
- Database: MongoDB or MySQL.
- API: REST endpoints for CRUD operations.
- Deployment: hosting server, domain, environment variables and database connection.

Full Stack Architecture



Layer	Responsibility
UI	Display forms, lists and controls
Client Logic	Validation and API calls
Backend	Routes, business logic and security
Database	Persistent storage
Deployment	Make app available online

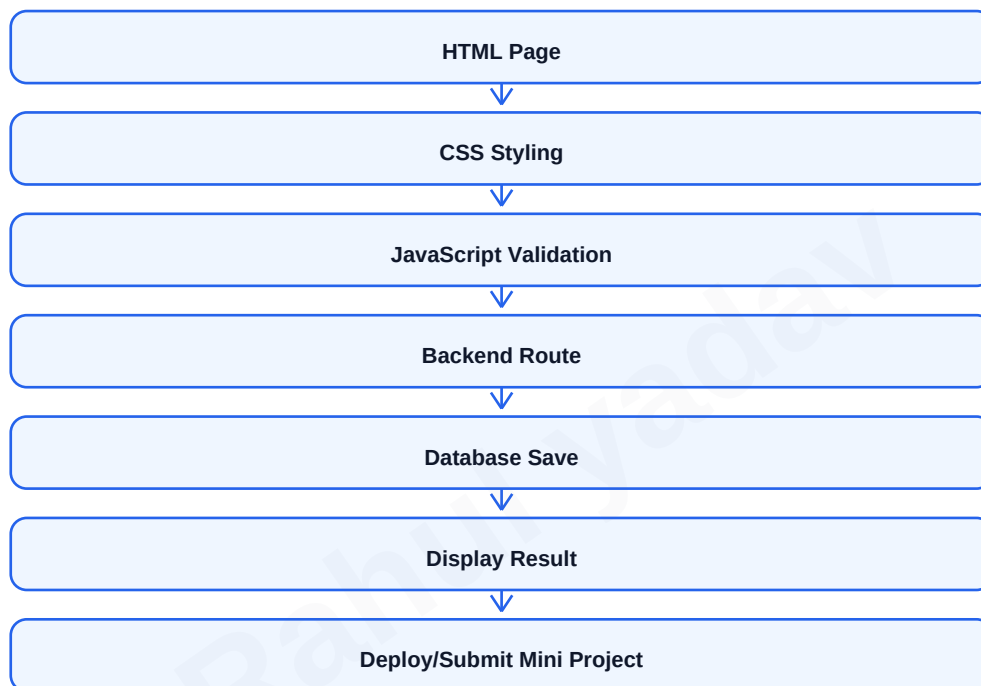
Suggested Lab Work and Mini Project Flow

Lab work should be done in sequence: static page using HTML, styling using CSS, dynamic behavior using JavaScript, enhanced UI using jQuery/Bootstrap, backend using Node.js/PHP and database using MongoDB.

A good mini project should include at least one form, validation, database storage, listing page and update/delete operations. Examples are student record system, notes app, contact manager, event registration or simple e-commerce catalog.

For viva, be ready to explain how request travels from browser to server and how data is saved in database.

- Create folder structure: index.html, style.css, app.js and backend files.
- Use semantic HTML and external CSS.
- Add JavaScript validation and DOM updates.
- Create Express/PHP backend route for form submission.
- Store data in MongoDB/database.
- Test with different inputs and browser sizes.



Mini Project Feature	Technology
Static pages	HTML5
Responsive design	CSS/flexbox/media queries
Validation	JavaScript/jQuery
Backend	Node.js/Express or PHP
Database	MongoDB/MySQL
Hosting	Shared/cloud/static hosting

Quick Revision and Important Exam Questions

Important long-answer questions include HTTP request-response, HTML document structure, forms and methods, HTML5 features, CSS selectors and box model, CSS positioning, JavaScript events, arrays and objects, jQuery features, Node.js architecture, Express routing, MongoDB overview, PHP form processing and website hosting.

For code-based answers, write short correct examples. For comparison questions, use tables. For architecture questions, draw request-response or full-stack diagram.

For practical exams, focus on creating a complete mini web page with form, CSS styling and JavaScript validation.

- HTML: tags, forms, GET/POST, HTML5 doctype and features.
- CSS: types, selectors, properties, box model and menu design.
- JavaScript: variables, operators, loops, events, arrays, objects, functions.
- jQuery: syntax, selectors, events, form UI.
- Node.js: modules, HTTP, Express and npm.
- MongoDB: database, collection, document and CRUD.
- PHP: syntax, arrays, functions and form processing.
- Web hosting: domain, server, FTP, CMS and full stack flow.

Question Type	Best Answer Pattern
HTTP	Diagram + request/response parts + methods/status
HTML Form	Controls + attributes + GET/POST + code
CSS	Selector/table + example
JavaScript	Theory + small code
Node/Express	Flow diagram + server code
CMS/Hosting	Definition + types + comparison